

EXPERIMENT 3

Difference between CLI(Command line interface) and GUI(Graphical user interface) and VUI(voice user interface)

REG NO: 240701322

Name: Mohamed Humaid Zahir Hussain

AIM

To write a program in CLI, GUI, VUI to rename file in the system.

Command line Interface(CLI)

- A Text-based Interface (Command line interface) enables users to communicate with a computer using the command line, as opposed to using a graphical interface.
- Compact and Powerful – CLI is usually faster and more powerful than a GUI, particularly for expert users who know the correct commands.
- Automation & Scripting – The command-line interface is there to allow users to automate tasks with scripts, making the repetitive tasks easier.
- Lightweight and Resource Friendly – Compared to GUI application, CLI uses very little system resources which is why they are most suited for use on remote servers
example: Windows command prompt

IMPLEMENTATION

```
command_line_interface.py > ...
1  import os
2  import sys
3
4  def rename_file(old_name, new_name):
5      try:
6          os.rename(old_name, new_name)
7          print(f"File renamed from {old_name} to {new_name}")
8      except FileNotFoundError:
9          print(f"Error: {old_name} not found.")
10     except Exception as e:
11         print(f"An error occurred: {e}")
12
13 if __name__ == "__main__":
14     if len(sys.argv) != 3:
15         print("Usage: python rename_file_cli.py <old_filename> <new_filename>")
16     else:
17         rename_file(sys.argv[1], sys.argv[2])
```

OUTPUT:

```
PS C:\Users\moham\Downloads\UID EXP 3> python command_line_interface.py old.py new.py
File renamed from old.py to new.py
PS C:\Users\moham\Downloads\UID EXP 3> 
```

Graphical user interface(GUI)

- A GUI uses visual elements including icons, buttons, and windows which makes it more user-friendly for users to work with the system.
 - More Resource Intensive – GUI applications use more system resources (CPU, RAM, and GPU) in contrast to a Command Line Interface (CLI) that can degrade performance in low-end devices.
 - Limited Automation: GUI does not have automation and scripting abilities like the CLI and is thus more tedious for repetitive jobs.
- example: Windows, Android, iOS

IMPLEMENTATION

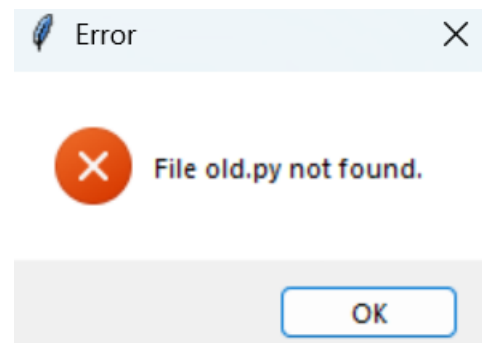
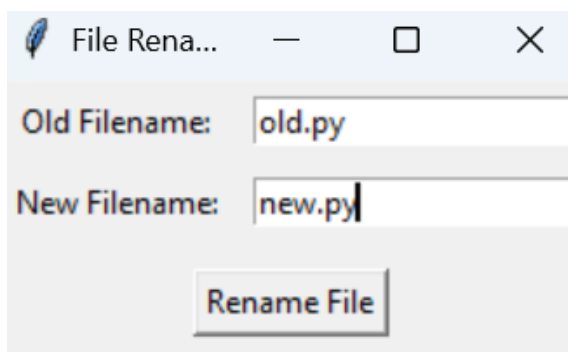
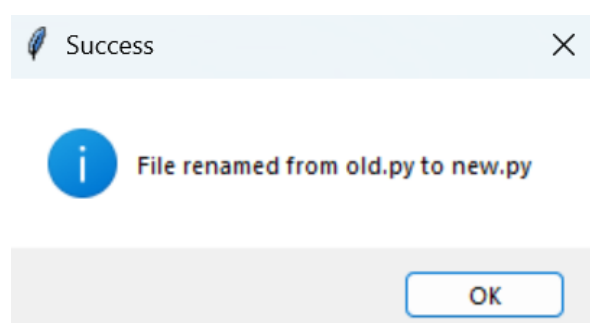
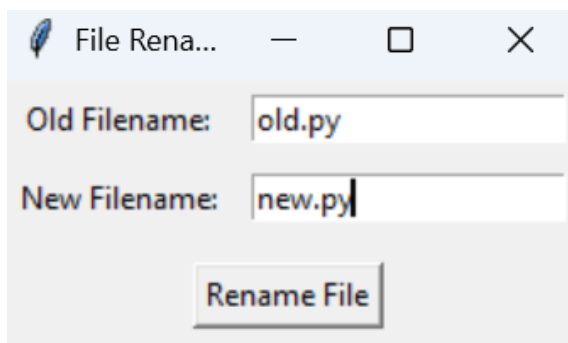
```
graphical_user_interface.py > ...
1  import tkinter as tk
2  from tkinter import messagebox
3  import os
4
5  def rename_file():
6      old_name = old_filename_entry.get()
7      new_name = new_filename_entry.get()
8
9      try:
10         os.rename(old_name, new_name)
11         messagebox.showinfo(
12             "Success",
13             f"File renamed from {old_name} to {new_name}"
14         )
15     except FileNotFoundError:
16         messagebox.showerror(
17             "Error",
18             f"File {old_name} not found."
19         )
20     except Exception as e:
21         messagebox.showerror(
22             "Error",
23             f"An error occurred: {e}"
24         )
```

```

25
26 # Set up the main window
27 root = tk.Tk()
28 root.title("File Renamer")
29
30 # Create and place labels, entries, and buttons
31 tk.Label(root, text="Old Filename:").grid(row=0, column=0, padx=5, pady=5)
32 tk.Label(root, text="New Filename:").grid(row=1, column=0, padx=5, pady=5)
33
34 old_filename_entry = tk.Entry(root)
35 old_filename_entry.grid(row=0, column=1, padx=5, pady=5)
36
37 new_filename_entry = tk.Entry(root)
38 new_filename_entry.grid(row=1, column=1, padx=5, pady=5)
39
40 rename_button = tk.Button(root, text="Rename File", command=rename_file)
41 rename_button.grid(row=2, column=0, columnspan=2, pady=10)
42
43 # Start the Tkinter event loop
44 root.mainloop()

```

OUTPUT:



Voice user interface(VUI)

- No Hands – VUI allows me to talk to a system and in some cases this is much more convenient than having to type.
- Natural Communication– VUI allows for more natural and fluid communication, as it can process users' speech and verbal commands, eliminating typing or clicking.
- Accuracy Challenges – Speech recognition can occasionally have difficulty with accents, background noise, and complex commands, sometimes resulting in misinterpretation.

example: Amazon, Alexa, Google Assistant

IMPLEMENTATION

```
voice_user_interface.py > ...
1  import speech_recognition as sr
2  import os
3
4  recognizer = sr.Recognizer()
5  mic = sr.Microphone()
6
7
8  def listen():
9      with mic as source:
10         recognizer.adjust_for_ambient_noise(source)
11         audio = recognizer.listen(source)
12
13         try:
14             text = recognizer.recognize_google(audio)
15             return text.lower()
16         except sr.UnknownValueError:
17             print("❌ Sorry, I couldn't understand.")
18             return None
19         except sr.RequestError as e:
20             print(f"❌ Speech Recognition service error: {e}")
21             return None
22
23
24 def voice_file_renamer():
25     print("🔊 Welcome to the Voice-Controlled File Renamer!")
26
```

```

27     # Ask for old filename
28     print("👂 Say the name of the file you want to rename (without .txt)")
29     old_name = listen()
30
31     if old_name is None:
32         return
33
34     print(f"🗣️ You said: {old_name}")
35
36     # Ask for new filename
37     print("👂 Say the new name for the file (without .txt)")
38     new_name = listen()
39
40     if new_name is None:
41         return
42
43     print(f"🗣️ You said: {new_name}")
44
45     # Add .txt automatically
46     old_file = old_name + ".txt"
47     new_file = new_name + ".txt"
48

```

```

49     try:
50         os.rename(old_file, new_file)
51         print(f"✅ File successfully renamed from '{old_file}' to '{new_file}'")
52     except FileNotFoundError:
53         print(f"❌ File '{old_file}' not found.")
54     except Exception as e:
55         print(f"❌ An error occurred: {e}")
56
57
58 if __name__ == "__main__":
59     voice_file_renamer()

```

OUTPUT:

```

PS C:\Users\moham\Downloads\UID EXP 3> python -u "c:\Users\moham\Downloads\UID EXP 3\voice_user_interface.py"
👂 Welcome to the Voice-Controlled File Renamer!
👂 Say the name of the file you want to rename (without .txt)
🗣️ You said: old
👂 Say the new name for the file (without .txt)
🗣️ You said: sample
✅ File successfully renamed from 'old.txt' to 'sample.txt'
PS C:\Users\moham\Downloads\UID EXP 3>

```

User Satisfaction Comparison

To assess user satisfaction, you can compare the interfaces on the following aspects:

4. Efficiency (Time to complete task):

- CLI: Most efficient for advanced users.
- GUI: Moderate efficiency, dependent on visual navigation.
- VUI: Dependent on speech recognition accuracy and environmental conditions.

2. Learning Curve:

- CLI: Steep for non-technical users.
- GUI: Easy to learn for most users.
- VUI: Very shallow, as it mimics natural language commands, but voice accuracy issues might hinder this.

3. Error Handling:

- CLI: Minimal feedback, error prone for typos.
- GUI: Provides visual feedback with error messages.
- VUI: Dependent on recognition; can misunderstand commands if not clear.

4. User Satisfaction:

- CLI: Likely to be rated low by non-technical users, but advanced users will be very satisfied.
- GUI: Likely to be rated highly by most users due to ease of use.
- VUI: Satisfaction depends on voice recognition accuracy; users may find it convenient for hands-free tasks, but errors can cause frustration.

IMPLEMENTATION

```
satisfication.py > ...
1  def get_rating(interface_name):
2      while True:
3          try:
4              rating = int(input(f"{interface_name} (1-5): "))
5              if 1 <= rating <= 5:
6                  return rating
7              else:
8                  print("Please enter a number between 1 and 5.")
9          except ValueError:
10             print("Invalid input. Please enter a number.")
11
12
13  def survey():
14      print("Rate your satisfaction with the following interfaces (1-5):")
15
16      cli_satisfaction = get_rating("CLI")
17      gui_satisfaction = get_rating("GUI")
18      vui_satisfaction = get_rating("VUI")
19
20      print("\nYour satisfaction ratings:")
21      print(f"CLI: {cli_satisfaction}")
22      print(f"GUI: {gui_satisfaction}")
23      print(f"VUI: {vui_satisfaction}")
24
25
26  if __name__ == "__main__":
27      survey()
28
```

OUTPUT

```
PS C:\Users\moham\Downloads\UID EXP 3> python -u "c:\Users\moham\Downloads\UID EXP 3\satisfication.py"
Rate your satisfaction with the following interfaces (1-5):
CLI (1-5): 1
GUI (1-5): 4
VUI (1-5): 2

Your satisfaction ratings:
CLI: 1
GUI: 4
VUI: 2
PS C:\Users\moham\Downloads\UID EXP 3>
```

Result:

The CLI, GUI, and VUI implementations successfully renamed files and displayed appropriate success or error messages, demonstrating different user interaction methods for the same task.